Travaux Pratiques 1 — Introduction à la programmation de la carte UCA

Compte rendu

NOM Prénom — NOM Prénom · Groupe : ……

16 octobre 2026

Introduction

Ce TP sert à prendre en main la carte **UCA** (RFThings, microcontrôleur ATmega328PB). On la programme en C/C++ avec l'Arduino IDE. On utilise la LED bleue et le port série (UART) en sortie, puis en entrée. On lit ensuite les capteurs intégrés : SHTC3 (température et humidité), LTR-303A (lumière) et KXTJ3-1057 (accéléromètre). Enfin, on pilote les 21 LEDs RGB WS2812. La dernière partie combine ces éléments dans plusieurs programmes autonomes.

Tous les programmes de ce compte rendu ont été compilés pour la cible **RFThings UCA – version 3.9 et plus récente (ATmega328PB)** sans erreur ni dépassement mémoire (voir le tableau en annexe).

1. Installation et configuration de l'Arduino IDE

Étape	Action réalisée
1.1	Installation de l'Arduino IDE 2.x (Windows 10/11 64 bits, ou archive ZIP si l'on n'a pas les droits administrateur).
1.2	*File → Preferences → Additional boards manager URLs* : `https://rfthings.github.io/ArduinoBoardManagerJSON/package_rfthings-avr_index.json`, puis installation du package **RFThings AVR Boards by RFThings Vietnam** (v0.1.1-rc6) dans le gestionnaire de cartes.
1.3	Installation du pilote **CH340** (puce de conversion USB ↔ série de la carte).
1.4	*Tools → Board → RFThings AVR Boards → RFThings UCA, Board Version : 3.9 and newer (ATmega328PB)*, puis choix du port COM de la carte.
1.5	Installation de la bibliothèque **FastLED**

Étape	Action réalisée
	(Library Manager).
1.6	Test avec *File → Examples → 01.Basics → Blink* : la LED bleue clignote.

Question : à quoi sert le pilote CH340 ? L'ATmega328PB n'a pas de port USB natif. La puce CH340C convertit l'USB en liaison série (UART). Le PC voit alors un port COM virtuel, utilisé pour téléverser le programme (via le bootloader) et pour le moniteur série. Sans ce pilote, aucun port COM n'apparaît dans *Tools → Port*.

Question : pourquoi choisir la « Board Version » ? Les cartes UCA 3.8 et antérieures utilisent un ATmega328P, les 3.9 et suivantes un **ATmega328PB**. Ces deux microcontrôleurs n'ont pas exactement les mêmes périphériques ni le même brochage. Le compilateur doit donc connaître le bon modèle (`-mmcu`) et le bon fichier de brochage (*variant*).

Dans le *variant* UCA, `LED_BUILTIN` est défini à la broche **13**. C'est la LED bleue de la carte.

2. Programmation Arduino

Interface de l'IDE

N°	Élément	Rôle
1	Vérifier	Compile le sketch
2	Téléverser	Compile puis programme la carte
3	Débogage	Débogage pas à pas (non disponible sur AVR sans sonde)
4	Sélecteur carte/port	Choix de la carte et du port COM
5	Traceur série	Trace graphiquement les valeurs reçues
6	Moniteur série	Affiche le texte envoyé par `Serial.print()`
7–11	Barre latérale	Carnet de croquis, gestionnaire de cartes, de bibliothèques, débogueur, recherche
12	Éditeur	Code source C/C++ (`.ino`)
13	Zone de messages	Résultat de compilation et de téléversement

Structure d'un sketch (Blink)

- `setup()` est exécutée **une seule fois**, à la mise sous tension ou au reset. Elle sert aux initialisations. Ici, `pinMode(LED_BUILTIN, OUTPUT)` configure la broche 13 en sortie.
- `loop()` est une **boucle infinie** : son contenu est répété sans fin.
- `digitalWrite(broche, HIGH/LOW)` met la broche à 1 (5 V ou 3,3 V) ou à 0 (0 V).
- `delay(ms)` bloque le programme pendant le nombre de millisecondes donné.

```
// Étape 1.6 — Test de clignotement de la LED bleue (LED_BUILTIN = D13 sur la carte UCA)
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);     // LED_BUILTIN en sortie
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);  // LED allumée
  delay(1000);                      // 1 s
  digitalWrite(LED_BUILTIN, LOW);   // LED éteinte
  delay(1000);                      // 1 s
}
```

Résultat : la LED est allumée 1 s puis éteinte 1 s. Sa période est donc de 2 s, soit une fréquence de 0,5 Hz.

Étape 2.1 — Sortie série (Blink_Mod)

```
// Étape 2.1 — Sortie série : Blink + écriture de l'état de la LED sur l'UART
void setup() {
  Serial.begin(9600);               // UART à 9600 bauds
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.println("Initialisation OK");
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  Serial.println(1);                // "1" -> visible dans le moniteur et le traceur série
  delay(1000);

  digitalWrite(LED_BUILTIN, LOW);
  Serial.println(0);
  delay(1000);
}
```

- `Serial.begin(9600)` initialise l'UART à **9600 bauds** (bits par seconde). Une trame 8N1 fait 10 bits par caractère, ce qui donne environ 960 caractères/s.
- `Serial.println(x)` envoie la valeur en **texte ASCII**, suivie de `\r` (ASCII 13) et `\n` (ASCII 10).

- Le **moniteur série** affiche « Initialisation OK » puis alterne `1` et `0` chaque seconde. Le **traceur série** affiche un signal carré entre 0 et 1, de période 2 s.

Question : pourquoi le moniteur doit-il être à 9600 bauds ? L'UART est asynchrone : il n'y a pas de fil d'horloge. L'émetteur et le récepteur doivent donc utiliser le même débit. Sinon, le récepteur échantillonne les bits au mauvais moment et affiche des caractères incohérents.

Étape 2.2 — Entrée série

```
// Étape 2.2 — Entrée série : l'utilisateur saisit le délai de
clignotement (0..9 s)
// Version du TP, avec en plus la vérification que le caractère reçu
est bien un chiffre.
unsigned long blinkDelay = 1000;

void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);

  while (true) {
    Serial.println("Please enter the blink delay (0..9) in (s): ");
    while (!Serial.available());          // attente d'un caractère
    char c = Serial.read();
    while (Serial.available()) Serial.read(); // vide le buffer (\r, \
n…)
    if (c >= '0' && c <= '9') {
      blinkDelay = (c - '0') * 1000UL;    // ASCII -> nombre, puis en
ms
      break;
    }
    Serial.println("Entree invalide, recommencez.");
  }
  Serial.print("Delai = ");
  Serial.print(blinkDelay);
  Serial.println(" ms");
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(blinkDelay);
  digitalWrite(LED_BUILTIN, LOW);
  delay(blinkDelay);
}
```

- `Serial.available()` renvoie le **nombre d'octets** en attente dans le buffer de réception. La valeur est 0 si le buffer est vide. *Remarque* : c'est `Serial.read()` qui renvoie −1 quand aucun octet n'est disponible, pas `available()` comme l'indique l'énoncé.
- `while(!Serial.available());` est une **attente active** : on reste bloqué tant que l'utilisateur n'a rien envoyé.

- `Serial.read() - '0'` : le caractère reçu est un code ASCII. Par exemple, `'5'` vaut 53 et `'0'` vaut 48, donc 53 – 48 = **5**. Comme les chiffres `'0'`…`'9'` ont des codes consécutifs (48…57), cette soustraction convertit n'importe quel chiffre en sa valeur numérique.
- Par rapport à l'énoncé, on a ajouté une **vérification de l'entrée** : si le caractère n'est pas entre `'0'` et `'9'`, on redemande. On vide aussi le buffer pour supprimer les `\r` ou `\n` ajoutés par le moniteur selon l'option « Nouvelle ligne ». Sans cette vérification, une lettre comme `'a'` (97) donnerait 49, soit un délai de 49 s.

Question : comment lire un nombre de plusieurs chiffres (ex. 12) ? On lit **tous** les caractères disponibles dans une boucle `while (Serial.available())`. On construit le nombre avec `nombre = nombre × 10 + chiffre` et on le valide à la réception de la fin de ligne. Cette version lit la saisie **pendant** que la LED clignote (lecture dans `loop()`, clignotement non bloquant avec `millis()`) :

```
// Étape 2.2 (suite) — Lecture d'un nombre de plusieurs chiffres (ex. "12") avec une boucle while,
// pendant que le programme tourne (lecture dans loop()). Délai saisi en millisecondes.
unsigned long blinkDelay = 1000;
unsigned long previousMillis = 0;
long saisie = 0;
bool chiffreRecu = false;

void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.println("Entrez un delai en ms puis Entree :");
}

void loop() {
  // Lecture caractère par caractère tant que le buffer n'est pas vide
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c >= '0' && c <= '9') {
      saisie = saisie * 10 + (c - '0');
      chiffreRecu = true;
    } else if ((c == '\n' || c == '\r') && chiffreRecu) {
      blinkDelay = saisie;
      Serial.print("Nouveau delai : ");
      Serial.print(blinkDelay);
      Serial.println(" ms");
      saisie = 0;
      chiffreRecu = false;
    }
  }

  // Clignotement non bloquant avec millis()
  if (millis() - previousMillis >= blinkDelay) {
    previousMillis = millis();
```

```
    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
  }
}
```

3. Fonctions intégrées — `millis()`

```
// Étape 3.1 — Clignotement non bloquant avec millis()
const unsigned long interval = 1000;  // ms

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  static unsigned long previousMillis = 0;
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));  // bascule
l'état de la LED
  }
}
```

- `millis()` renvoie le nombre de millisecondes écoulées depuis le démarrage. La valeur est un `unsigned long` sur 32 bits, comptée par l'interruption du Timer0.
- `previousMillis` mémorise l'instant du dernier changement d'état. Le mot-clé `static` conserve sa valeur entre deux appels de `loop()`.
- Quand `currentMillis - previousMillis >= interval`, on bascule la LED avec `!digitalRead(...)` et on met `previousMillis` à jour.

Question : quel est l'avantage par rapport à `delay()` ? `delay()` **bloque** le microcontrôleur, qui ne peut rien faire d'autre (lire un capteur, répondre au port série…). Avec `millis()`, `loop()` tourne en permanence et ne fait que *vérifier* si l'intervalle est écoulé. Le programme reste **non bloquant**, ce qui permet de gérer plusieurs tâches avec des périodes différentes.

Remarque : `millis()` revient à 0 après 2^{32} ms, soit environ 49,7 jours. L'écriture `currentMillis - previousMillis` en arithmétique non signée reste juste même au moment de ce débordement. Écrire `currentMillis >= previousMillis + interval` serait faux à ce moment-là.

4. Acquisition des données des capteurs

Bibliothèques installées : **arduino-sht** (Sensirion, v2.0.0), **Adafruit LTR329 and LTR303** (v2.0.1, avec Adafruit BusIO en dépendance), **KXTJ3-1057** (v0.1.1) et **FastLED** (v3.10.7).

Les trois capteurs partagent le **même bus I2C** (fils SDA/SCL). Chacun y est identifié par son **adresse** : SHTC3 = 0x70, LTR-303 = 0x29, KXTJ3-1057 = 0x0E.

4.2 — SHTC3 (température / humidité)

```
// Étape 4.2 — Capteur de température et d'humidité SHTC3 (I2C)
#include <Wire.h>
#include "SHTSensor.h"

SHTSensor sht;
// Pour forcer un modèle précis : SHTSensor sht(SHTSensor::SHTC3);

void setup() {
  Wire.begin();
  Serial.begin(9600);
  delay(1000);  // laisse le temps au moniteur série de s'ouvrir

  if (sht.init()) {
    Serial.print("init(): success\n");
  } else {
    Serial.print("init(): failed\n");
  }
}

void loop() {
  if (sht.readSample()) {
    Serial.print("SHT:\n");
    Serial.print("  RH: ");
    Serial.print(sht.getHumidity(), 2);
    Serial.print("\n");
    Serial.print("  T:  ");
    Serial.print(sht.getTemperature(), 2);
    Serial.print("\n");
  } else {
    Serial.print("Error in readSample()\n");
  }
  delay(1000);
}
```

- `Wire.begin()` initialise l'I2C en maître.
- `sht.init()` sonde le bus, détecte le modèle SHT présent (SHTC3 ici) et l'initialise. Elle renvoie `true` en cas de succès.
- `sht.readSample()` déclenche une mesure, lit les octets bruts et vérifie leur CRC. Elle les convertit ensuite en °C et en %HR.
- `sht.getTemperature()` et `sht.getHumidity()` renvoient les dernières valeurs converties (`float`). Le 2e argument de `Serial.print(val, 2)` fixe le nombre de décimales à 2.
- Sortie attendue à 9600 bauds : `SHT:  RH: 60.81  T: 24.27`, une mesure par seconde.

4.3 — LTR-303A (lumière)

```
// Étape 4.3 — Capteur de lumière LTR-303A (I2C) — moniteur série à
115200 bauds
#include "Adafruit_LTR329_LTR303.h"

Adafruit_LTR303 ltr = Adafruit_LTR303();

void setup() {
  Serial.begin(115200);
  while (!Serial) {
    delay(10);
  }
  Serial.println("Adafruit LTR-303 simple test");

  if (!ltr.begin()) {
    Serial.println("Couldn't find LTR sensor!");
    while (1) delay(10);
  }
  Serial.println("Found LTR sensor!");

  ltr.setGain(LTR3XX_GAIN_1);                    // gain x1
  ltr.setIntegrationTime(LTR3XX_INTEGTIME_50);  // intégration 50 ms
  ltr.setMeasurementRate(LTR3XX_MEASRATE_50);   // une mesure toutes
les 50 ms
}

void loop() {
  bool valid;
  uint16_t visible_plus_ir, infrared;

  if (ltr.newDataAvailable()) {
    valid = ltr.readBothChannels(visible_plus_ir, infrared);
    if (valid) {
      Serial.print("CH0 Visible + IR: ");
      Serial.print(visible_plus_ir);
      Serial.print("\t\tCH1 Infrared: ");
      Serial.println(infrared);
    }
  }
  delay(1000);
}
```

- `setGain(LTR3XX_GAIN_1)` règle le gain à ×1, la plage la plus large (jusqu'à environ 64 klux). `setIntegrationTime(50 ms)` fixe la durée de comptage d'une mesure. `setMeasurementRate(50 ms)` fixe la période entre deux mesures.
- `newDataAvailable()` indique si une nouvelle mesure est prête. `readBothChannels(ch0, ch1)` lit le **canal 0 (visible + IR)** et le **canal 1 (IR seul)**.
- La lumière visible correspond à peu près à CH0 − CH1. Ces valeurs sont des **comptes bruts sur 16 bits**, pas des lux. Pour obtenir des lux, la datasheet du LTR-303 donne une

formule qui dépend du rapport CH1/(CH0+CH1), du gain et du temps d'intégration. Nous l'avons utilisée dans la partie 5 (fonction `calculLux()`).

- Le moniteur série doit être réglé sur **115200 bauds**.

4.4 — KXTJ3-1057 (accéléromètre 3 axes)

```
// Étape 4.4 — Accéléromètre 3 axes KXTJ3-1057 (I2C, adresse 0x0E) — 115200 bauds
#include "kxtj3-1057.h"

float sampleRate = 6.25;  // Hz : 0.781, 1.563, 3.125, 6.25, 12.5, 25, 50, 100, 200, 400, 800, 1600
uint8_t accelRange = 2;   // ±2, 4, 8 ou 16 g
bool highRes = false;     // mode haute résolution

KXTJ3 myIMU(0x0E);        // adresse 0x0E ou 0x0F

void setup() {
  Serial.begin(115200);
  delay(100);

  if (myIMU.begin(sampleRate, accelRange, highRes) == IMU_SUCCESS) {
    Serial.println("IMU initialized.");
  } else {
    Serial.println("Failed to initialize IMU.");
    while (true);
  }
}

void loop() {
  myIMU.standby(false);  // sort l'IMU du mode veille

  uint8_t dataLowRes = 0;

  if (myIMU.readRegister(&dataLowRes, KXTJ3_XOUT_H) == IMU_SUCCESS) {
    Serial.print(" Acceleration X float = ");
    Serial.println(myIMU.axisAccel(X), 4);
  }
  if (myIMU.readRegister(&dataLowRes, KXTJ3_YOUT_H) == IMU_SUCCESS) {
    Serial.print(" Acceleration Y float = ");
    Serial.println(myIMU.axisAccel(Y), 4);
  }
  if (myIMU.readRegister(&dataLowRes, KXTJ3_ZOUT_H) == IMU_SUCCESS) {
    Serial.print(" Acceleration Z float = ");
    Serial.println(myIMU.axisAccel(Z), 4);
  }

  myIMU.standby(true);   // remet l'IMU en veille
  delay(1000);
}
```

- Paramètres : 6,25 échantillons/s, plage ±2 g, résolution normale, adresse I2C **0x0E**.
- `standby(false)` sort le capteur de veille pour la mesure, et `standby(true)` l'y remet pour économiser l'énergie.
- `readRegister(&dataLowRes, KXTJ3_ZOUT_H)` lit un registre de 8 bits. `&dataLowRes` est l'**adresse** de la variable : la fonction y écrit la valeur lue (passage par pointeur). Ici, l'appel sert surtout à **tester la communication**, car `axisAccel(Z)` relit elle-même les registres `ZOUT_L/ZOUT_H` sur 16 bits et les convertit en **g**.
- **Observation :** carte posée à plat, on lit X ≈ 0, Y ≈ 0 et **Z ≈ 1 g**. Un accéléromètre mesure l'accélération *propre* : au repos, il mesure la réaction du support à la pesanteur, soit 1 g vers le haut. En inclinant la carte, la gravité se répartit sur X et Y. En la secouant, les valeurs varient brusquement.

4.5 — LEDs RGB (WS2812 × 21)

```
// Étape 4.5 — 21 LEDs RGB (WS2812) — effet « cylon » multicolore — 57600 bauds
#include <FastLED.h>

#define NUM_LEDS 21
#define DATA_PIN 4
#define CLOCK_PIN 13   // inutile pour des WS2812 (une seule ligne de données)

CRGB leds[NUM_LEDS];

void setup() {
  Serial.begin(57600);
  Serial.println("resetting");
  FastLED.addLeds<WS2812, DATA_PIN, RGB>(leds, NUM_LEDS);
  FastLED.setBrightness(84);
}

void fadeall() {
  for (int i = 0; i < NUM_LEDS; i++) {
    leds[i].nscale8(250);  // multiplie chaque composante par 250/256
  }
}

void loop() {
  static uint8_t hue = 0;
  Serial.print("x");
  for (int i = 0; i < NUM_LEDS; i++) {          // aller
    leds[i] = CHSV(hue++, 255, 255);
    FastLED.show();
    fadeall();
    delay(10);
  }
  Serial.print("x");
  for (int i = NUM_LEDS - 1; i >= 0; i--) {     // retour
```

```
    leds[i] = CHSV(hue++, 255, 255);
    FastLED.show();
    fadeall();
    delay(10);
  }
}
```

- `CRGB leds[21]` est le tableau en mémoire : une couleur R, G, B de 3 octets par LED. Rien n'est envoyé aux LEDs avant l'appel à `FastLED.show()`.
- `addLeds<WS2812, DATA_PIN, RGB>` déclare le type de LED et la broche de données (D4). Les WS2812 n'utilisent qu'**un seul fil de données**, donc `CLOCK_PIN` est inutile. `setBrightness(84)` limite la luminosité globale à 84/255, soit environ 33 %.
- `CHSV(hue, 255, 255)` décrit une couleur par sa teinte (*hue*), avec saturation et luminosité maximales. `hue++` fait défiler l'arc-en-ciel.
- `fadeall()` applique `nscale8(250)` à chaque LED, ce qui multiplie chaque composante par 250/256 (≈ 0,977) à chaque pas. Les LEDs déjà allumées s'estompent progressivement et forment une « traînée ». L'aller-retour produit l'effet « cylon » multicolore.

5. Partie non guidée

Choix communs :

- La « luminosité en lux » est calculée avec la formule de la datasheet LTR-303 (gain ×1, intégration 100 ms, fonction `calculLux()`).
- Les capteurs sont lus de façon **non bloquante** avec `millis()`.
- Moniteur série à **115200 bauds**.

I.1 — Seuil de luminosité

Si la luminosité dépasse 500 lux, les 21 LEDs passent au rouge et un message s'affiche. Sinon, elles sont éteintes.

```
// Partie 5.I — Seuil de luminosité
// Les 21 LEDs RGB s'allument en rouge quand la luminosité ambiante
dépasse 500 lux,
// et un message est affiché sur le moniteur série (115200 bauds).
#include <FastLED.h>
#include "Adafruit_LTR329_LTR303.h"

#define NUM_LEDS 21
#define DATA_PIN 4
#define SEUIL_LUX 500.0

CRGB leds[NUM_LEDS];
Adafruit_LTR303 ltr = Adafruit_LTR303();

// Conversion des comptes bruts en lux (formule de la datasheet LTR-
```

```
303, gain x1, intégration 100 ms)
float calculLux(uint16_t ch0, uint16_t ch1) {
  if (ch0 + ch1 == 0) return 0;
  float ratio = (float)ch1 / (ch0 + ch1);
  if (ratio < 0.45) return 1.7743 * ch0 + 1.1059 * ch1;
  if (ratio < 0.64) return 4.2785 * ch0 - 1.9548 * ch1;
  if (ratio < 0.85) return 0.5926 * ch0 + 0.1185 * ch1;
  return 0;
}

void setup() {
  Serial.begin(115200);
  FastLED.addLeds<WS2812, DATA_PIN, RGB>(leds, NUM_LEDS);
  FastLED.setBrightness(84);
  FastLED.clear(true);

  if (!ltr.begin()) {
    Serial.println("Capteur LTR-303 introuvable !");
    while (1) delay(10);
  }
  ltr.setGain(LTR3XX_GAIN_1);
  ltr.setIntegrationTime(LTR3XX_INTEGTIME_100);
  ltr.setMeasurementRate(LTR3XX_MEASRATE_500);
  Serial.println("Seuil de luminosite : pret");
}

void loop() {
  uint16_t visible_plus_ir, infrared;

  if (ltr.newDataAvailable() && ltr.readBothChannels(visible_plus_ir,
infrared)) {
    float lux = calculLux(visible_plus_ir, infrared);

    Serial.print("Visible+IR = ");
    Serial.print(visible_plus_ir);
    Serial.print("  IR = ");
    Serial.print(infrared);
    Serial.print("  Lux = ");
    Serial.println(lux, 1);

    if (lux > SEUIL_LUX) {
      fill_solid(leds, NUM_LEDS, CRGB::Red);
      Serial.println("Luminosite > 500 lux : LEDs rouges allumees");
    } else {
      fill_solid(leds, NUM_LEDS, CRGB::Black);
    }
    FastLED.show();
  }
  delay(500);
}
```

Pour tester : sous l'éclairage normal d'une salle (≈ 300 lux), les LEDs restent éteintes. En éclairant le capteur avec la lampe d'un téléphone, la valeur dépasse 500 lux, les LEDs deviennent rouges et le message s'affiche.

I.2 — Détection d'inclinaison

```
// Partie 5.I — Détection d'inclinaison
// X > 0.5 g  -> "Inclinée à droite"
// X < -0.5 g -> "Inclinée à gauche"
#include "kxtj3-1057.h"

#define SEUIL_G 0.5

KXTJ3 myIMU(0x0E);

void setup() {
  Serial.begin(115200);
  delay(100);
  // 50 Hz, plage ±2 g suffisante (on mesure la projection de la
gravité, |X| <= 1 g)
  if (myIMU.begin(50, 2, false) != IMU_SUCCESS) {
    Serial.println("Failed to initialize IMU.");
    while (true);
  }
  myIMU.standby(false);  // on laisse l'accéléromètre actif en continu
  Serial.println("IMU initialized.");
}

void loop() {
  float x = myIMU.axisAccel(X);

  Serial.print("X = ");
  Serial.print(x, 3);
  Serial.print(" g  -> ");

  if (x > SEUIL_G) {
    Serial.println("Inclinée à droite");
  } else if (x < -SEUIL_G) {
    Serial.println("Inclinée à gauche");
  } else {
    Serial.println("A plat");
  }
  delay(500);
}
```

Au repos, X ≈ 0 g, ce qui affiche « A plat ». Quand on penche la carte d'un angle θ autour de l'axe Y, on mesure $X = \sin(\theta)$ g. Le seuil de 0,5 g correspond donc à une inclinaison d'environ **30°**. Le signe de X donne le côté (droite ou gauche, selon l'orientation du capteur sur la carte).

II.1 — Station météo embarquée

```
// Partie 5.II — Station météo embarquée
// a. Affiche température, humidité et luminosité toutes les 5 s
// b. T > 30 °C et HR < 30 %  -> "Risque de sécheresse"
// c. Luminosité < 100 lux    -> "Nuit détectée"
#include <Wire.h>
#include "SHTSensor.h"
#include "Adafruit_LTR329_LTR303.h"

const unsigned long PERIODE = 5000;  // ms

SHTSensor sht;
Adafruit_LTR303 ltr = Adafruit_LTR303();

float calculLux(uint16_t ch0, uint16_t ch1) {
  if (ch0 + ch1 == 0) return 0;
  float ratio = (float)ch1 / (ch0 + ch1);
  if (ratio < 0.45) return 1.7743 * ch0 + 1.1059 * ch1;
  if (ratio < 0.64) return 4.2785 * ch0 - 1.9548 * ch1;
  if (ratio < 0.85) return 0.5926 * ch0 + 0.1185 * ch1;
  return 0;
}

void setup() {
  Wire.begin();
  Serial.begin(115200);
  delay(1000);

  if (!sht.init()) Serial.println("SHTC3 : init failed");
  if (!ltr.begin()) Serial.println("LTR-303 : introuvable");
  ltr.setGain(LTR3XX_GAIN_1);
  ltr.setIntegrationTime(LTR3XX_INTEGTIME_100);
  ltr.setMeasurementRate(LTR3XX_MEASRATE_500);

  Serial.println("Station meteo demarree");
}

void loop() {
  static unsigned long previousMillis = 0;
  if (millis() - previousMillis < PERIODE) return;  // non bloquant
  previousMillis = millis();

  Serial.println("----- Station meteo -----");

  bool shtOk = sht.readSample();
  float t = sht.getTemperature();
  float h = sht.getHumidity();
  if (shtOk) {
    Serial.print("Temperature : "); Serial.print(t, 2);
```

```
Serial.println(" C");
    Serial.print("Humidite    : "); Serial.print(h, 2);
Serial.println(" %");
  } else {
    Serial.println("Erreur lecture SHTC3");
  }

  uint16_t ch0, ch1;
  bool ltrOk = ltr.readBothChannels(ch0, ch1);
  float lux = calculLux(ch0, ch1);
  if (ltrOk) {
    Serial.print("Luminosite  : "); Serial.print(lux, 1);
Serial.println(" lux");
  } else {
    Serial.println("Erreur lecture LTR-303");
  }

  if (shtOk && t > 30.0 && h < 30.0) Serial.println("Risque de
sécheresse");
  if (ltrOk && lux < 100.0)          Serial.println("Nuit détectée");
}
```

- (a) L'affichage se fait toutes les 5 s grâce à `millis()` (période `PERIODE = 5000`), sans `delay()`.
- (b) « Risque de sécheresse » s'affiche si **T > 30 °C ET HR < 30 %** (opérateur &&).
- (c) « Nuit détectée » s'affiche si la luminosité est **< 100 lux**. Pour tester, il suffit de couvrir le capteur avec un doigt.

II.2 — Simulation d'une alerte IoT

```
// Partie 5.II — Simulation d'une alerte IoT
// a. Inclinaison dangereuse : |accélération| > 2 g sur n'importe quel
axe
// b. Surchauffe             : température > 40 °C
// c. Obscurité totale       : luminosité < 10 lux
// Format : "Alerte : [Type d'alerte] à [millis] ms"
// Une alerte est émise au moment où la condition devient vraie (front
montant),
// pour ne pas inonder le moniteur série tant que la condition reste
vraie.
#include <Wire.h>
#include "SHTSensor.h"
#include "Adafruit_LTR329_LTR303.h"
#include "kxtj3-1057.h"

SHTSensor sht;
Adafruit_LTR303 ltr = Adafruit_LTR303();
```

```
KXTJ3 myIMU(0x0E);

bool alerteChoc = false, alerteChaud = false, alerteNoir = false;

float calculLux(uint16_t ch0, uint16_t ch1) {
  if (ch0 + ch1 == 0) return 0;
  float ratio = (float)ch1 / (ch0 + ch1);
  if (ratio < 0.45) return 1.7743 * ch0 + 1.1059 * ch1;
  if (ratio < 0.64) return 4.2785 * ch0 - 1.9548 * ch1;
  if (ratio < 0.85) return 0.5926 * ch0 + 0.1185 * ch1;
  return 0;
}

void envoyerAlerte(const char *type) {
  Serial.print("Alerte : ");
  Serial.print(type);
  Serial.print(" à ");
  Serial.print(millis());
  Serial.println(" ms");
}

// Émet l'alerte uniquement lors du passage faux -> vrai
void verifier(bool condition, bool &etat, const char *type) {
  if (condition && !etat) envoyerAlerte(type);
  etat = condition;
}

void setup() {
  Wire.begin();
  Serial.begin(115200);
  delay(1000);

  if (!sht.init()) Serial.println("SHTC3 : init failed");
  if (!ltr.begin()) Serial.println("LTR-303 : introuvable");
  ltr.setGain(LTR3XX_GAIN_1);
  ltr.setIntegrationTime(LTR3XX_INTEGTIME_100);
  ltr.setMeasurementRate(LTR3XX_MEASRATE_100);

  // Plage ±8 g : avec ±2 g la mesure sature à 2 g et le seuil "> 2 g"
ne serait jamais atteint
  if (myIMU.begin(100, 8, false) != IMU_SUCCESS) Serial.println("KXTJ3
: init failed");
  myIMU.standby(false);

  Serial.println("Systeme IoT pret");
}

void loop() {
  // Accéléromètre : lu toutes les 20 ms pour ne pas rater un choc
```

```
bref
  float ax = myIMU.axisAccel(X), ay = myIMU.axisAccel(Y), az =
myIMU.axisAccel(Z);
  verifier(fabs(ax) > 2.0 || fabs(ay) > 2.0 || fabs(az) > 2.0,
alerteChoc, "Inclinaison dangereuse");

  // Température et lumière : toutes les 500 ms
  static unsigned long previousMillis = 0;
  if (millis() - previousMillis >= 500) {
    previousMillis = millis();

    if (sht.readSample()) verifier(sht.getTemperature() > 40.0,
alerteChaud, "Surchauffe");

    uint16_t ch0, ch1;
    if (ltr.readBothChannels(ch0, ch1)) verifier(calculLux(ch0, ch1) <
10.0, alerteNoir, "Obscurité totale");
  }
  delay(20);
}
```

- Format respecté : `Alerte : Surchauffe à 15200 ms`. L'heure est la valeur de `millis()` au moment de la détection.
- **Plage de l'accéléromètre portée à ±8 g.** Avec la plage par défaut de ±2 g, la mesure **sature** à 2 g et la condition « > 2 g » ne serait jamais vraie.
- On compare la **valeur absolue** (`fabs`) pour détecter les deux sens sur chaque axe.
- Chaque alerte n'est émise **qu'au moment où la condition devient vraie** (détection de front montant avec une variable d'état). Le moniteur n'est donc pas inondé tant que la condition persiste, comme le ferait un vrai objet IoT qui limite ses envois.
- L'accéléromètre est lu toutes les 20 ms pour ne pas rater un choc bref. La température et la lumière sont lues toutes les 500 ms.

II.3 — Clignotement synchronisé avec l'accélération

```
// Partie 5.II — Clignotement des LEDs RGB synchronisé avec
l'accéléromètre
// X > 1 g -> rouge, Y > 1 g -> vert, Z > 1 g -> bleu (les couleurs
s'additionnent si plusieurs axes)
#include <FastLED.h>
#include "kxtj3-1057.h"

#define NUM_LEDS 21
#define DATA_PIN 4
#define SEUIL_G 1.0
const unsigned long DEMI_PERIODE = 200;  // ms : 200 ms allumé / 200
ms éteint

CRGB leds[NUM_LEDS];
KXTJ3 myIMU(0x0E);
```

```ino
void setup() {
  Serial.begin(115200);
  FastLED.addLeds<WS2812, DATA_PIN, RGB>(leds, NUM_LEDS);
  FastLED.setBrightness(84);
  FastLED.clear(true);

  // Plage ±4 g pour pouvoir mesurer au-delà de 1 g sans saturer
  if (myIMU.begin(50, 4, false) != IMU_SUCCESS) {
    Serial.println("Failed to initialize IMU.");
    while (true);
  }
  myIMU.standby(false);
  Serial.println("IMU initialized.");
}

void loop() {
  static unsigned long previousMillis = 0;
  static bool allume = false;

  float x = myIMU.axisAccel(X);
  float y = myIMU.axisAccel(Y);
  float z = myIMU.axisAccel(Z);

  CRGB couleur = CRGB::Black;
  if (x > SEUIL_G) couleur += CRGB::Red;
  if (y > SEUIL_G) couleur += CRGB::Green;
  if (z > SEUIL_G) couleur += CRGB::Blue;

  if (millis() - previousMillis >= DEMI_PERIODE) {
    previousMillis = millis();
    allume = !allume;

    fill_solid(leds, NUM_LEDS, allume ? couleur : CRGB::Black);
    FastLED.show();

    Serial.print("X="); Serial.print(x, 2);
    Serial.print(" Y="); Serial.print(y, 2);
    Serial.print(" Z="); Serial.println(z, 2);
  }
}
```

- X > 1 g donne du rouge, Y > 1 g du vert et Z > 1 g du bleu. Si plusieurs axes dépassent le seuil, les couleurs **s'additionnent** (par exemple rouge + bleu = magenta).
- Le clignotement (200 ms allumé, 200 ms éteint) est géré par `millis()`.
- La plage est réglée à ±4 g pour mesurer au-delà de 1 g sans saturer.
- **Remarque :** à plat, Z vaut environ 1 g à cause de la gravité. Avec le bruit de mesure, Z oscille autour du seuil et la LED bleue peut clignoter de façon irrégulière au repos. Pour éviter cela, on pourrait utiliser un seuil un peu plus haut (1,2 g) ou soustraire la gravité.

Conclusion

Ce TP nous a permis de :

- mettre en place toute la chaîne de développement de la carte UCA (IDE, package de carte, pilote USB et bibliothèques) ;
- comprendre la structure `setup()` / `loop()` ;
- utiliser l'UART en sortie (`Serial.print`) et en entrée (`Serial.available` / `Serial.read`, conversion ASCII) ;
- remplacer `delay()` par `millis()` pour écrire des programmes non bloquants ;
- exploiter les trois capteurs I2C et les LEDs RGB.

La partie non guidée montre comment combiner ces briques en petites applications, comme une station météo ou des alertes de type IoT. Elle a aussi mis en évidence deux points qui ne sont pas dans l'énoncé : la **saturation** de l'accéléromètre (choix de la plage de mesure) et la différence entre **comptes bruts et lux** pour le capteur de lumière.

Annexe — Résultats de compilation

Cible : `rfthings-avr:avr:uca`, version 3.9+ (ATmega328PB), package RFThings AVR Boards 0.1.1-rc6. Mémoire disponible : 30 720 octets de Flash et 2 048 octets de RAM.

Sketch	Flash (octets)	RAM (octets)
Etape_1_6_Blink	1158 (3 %)	9 (0 %)
Etape_3_1_Millis	1128 (3 %)	13 (0 %)
Etape_2_1_Sortie_Serie_Blink_Mod	2460 (8 %)	206 (10 %)
Etape_2_2_Entree_Serie_1_chiffre	2644 (8 %)	280 (13 %)
Etape_2_2_Entree_Serie_plusieurs_chiffres	2676 (8 %)	257 (12 %)
Partie_5_I_1_Seuil_Luminosite	13262 (43 %)	739 (36 %)
Partie_5_I_2_Detection_Inclinaison	8100 (26 %)	527 (25 %)
Partie_5_II_1_Station_Meteo	12850 (41 %)	738 (36 %)
Partie_5_II_2_Alerte_IoT	14290 (46 %)	660 (32 %)
Partie_5_II_3_Clignotement_Accelerometre	12000 (39 %)	657 (32 %)
Etape_4_4_UCA_Board_KXTJ3	8306 (27 %)	543 (26 %)
Etape_4_3_UCA_Board_L	8280 (26 %)	530 (25 %)

Sketch	Flash (octets)	RAM (octets)
TR303		
Etape_4_5_UCA_Board_RGBLED	5038 (16 %)	374 (18 %)
Etape_4_2_UCA_Board_SHTC3	8928 (29 %)	570 (27 %)